

Minimum Viable Product (MVP)

Motor Inteligente de Segmentación de Clientes

Empresa: PluriOne S.A. de C.V.

Nombre comercial: Develop Talent & Technology

Proyecto: Motor Inteligente de Segmentación de Clientes

Responsable del desarrollo: Alan Alejandro Betancourt Ramírez

Duración: 3 semanas

Versión: 1.0

1. Objetivo del MVP

Desarrollar una primera versión funcional del Motor Inteligente de Segmentación de Clientes que permita demostrar el funcionamiento principal del producto mediante datos simulados.

El MVP deberá ser capaz de recibir información estructurada de clientes, procesarla, generar segmentos mediante Machine Learning, almacenar los resultados y mostrarlos mediante una interfaz web.

Además, deberá incorporar recomendaciones comerciales básicas mediante Inteligencia Artificial Generativa.

2. Alcance del MVP

El MVP se enfocará únicamente en las funcionalidades necesarias para demostrar el valor principal del proyecto.

El sistema deberá permitir:

- trabajar con información simulada de clientes;
- importar o cargar un dataset estructurado;
- almacenar información relevante;
- preparar y limpiar los datos;
- calcular variables útiles para segmentación;
- ejecutar un modelo de Machine Learning;
- generar segmentos de clientes;
- consultar los segmentos;

- consultar los clientes pertenecientes a cada segmento;
 - visualizar información mediante una interfaz web;
 - generar recomendaciones comerciales para cada segmento;
 - exponer la información mediante una API.
-

3. Fuente de datos

Debido a que no se contará con acceso directo a los sistemas reales de la empresa, el MVP utilizará información simulada representativa de un CRM.

El dataset deberá contener información suficiente para demostrar diferentes tipos de segmentación.

Podrá incluir campos como:

- ID del cliente.
- Edad.
- Ciudad.
- Fecha de registro.
- Número de compras.
- Gasto total.
- Ticket promedio.
- Fecha de última compra.
- Categoría favorita.
- Número de clics en campañas.
- Correos abiertos.
- Visitas registradas.
- Carritos abandonados.

El dataset podrá ser proporcionado mediante CSV o JSON.

4. Importación de datos

El sistema deberá permitir cargar información estructurada de clientes.

Como mínimo, el MVP deberá aceptar:

- archivos CSV;
- archivos JSON.

Los datos deberán ser validados antes de ser utilizados.

En caso de encontrar registros inválidos, el sistema deberá evitar que estos afecten el proceso completo.

5. Base de datos

PostgreSQL será utilizado para almacenar la información principal del sistema.

Las entidades mínimas serán:

Clientes

Información general y comercial de cada cliente.

Segmentos

Información relacionada con los grupos generados por el modelo.

Resultados de segmentación

Relación entre clientes y segmentos.

Recomendaciones

Recomendaciones generadas para cada segmento.

6. Procesamiento de datos

Antes de utilizar los datos en el modelo, el sistema deberá realizar un proceso básico de preparación.

Este proceso incluirá:

- eliminación de duplicados;
- manejo de valores faltantes;
- conversión de tipos de datos;
- transformación de fechas;
- generación de variables necesarias;
- normalización o escalado cuando sea necesario.

El procesamiento será implementado principalmente mediante Python.

7. Variables iniciales

El MVP utilizará principalmente variables relacionadas con comportamiento comercial.

Como mínimo:

- días desde la última compra;
- número de compras;
- gasto total.

Estas variables representan el enfoque RFM:

Recency: cuánto tiempo ha pasado desde la última compra.

Frequency: cuántas compras ha realizado.

Monetary: cuánto dinero ha gastado.

También podrán utilizarse variables adicionales como:

- ticket promedio;
 - categoría favorita;
 - interacción con campañas;
 - visitas;
 - clics.
-

8. Modelo de Machine Learning

El MVP deberá implementar un modelo de Machine Learning desarrollado en Python.

Se utilizará inicialmente un algoritmo de clustering como K-Means.

El modelo deberá:

- recibir datos preparados;
- identificar grupos de clientes similares;
- asignar un cluster a cada cliente;
- permitir analizar las características de cada cluster.

Ejemplo:

- Cluster 0.
- Cluster 1.

- Cluster 2.
- Cluster 3.

Posteriormente, cada cluster deberá interpretarse comercialmente.

Ejemplo:

- Clientes de alto valor.
- Clientes frecuentes.
- Clientes ocasionales.
- Clientes con riesgo de inactividad.

Los nombres deberán basarse en las características reales de los datos simulados.

9. Validación del modelo

El modelo deberá probarse antes de integrarse completamente con el sistema.

Se deberán revisar aspectos como:

- coherencia de los grupos;
- distribución de clientes;
- separación entre clusters;
- estabilidad de los resultados.

Podrán utilizarse métricas como:

- Silhouette Score.
- Inertia.

El objetivo no será únicamente generar grupos, sino obtener segmentos que puedan interpretarse de manera útil.

10. Azure Machine Learning

Una vez validado el modelo localmente, se deberá registrar y desplegar mediante Azure Machine Learning.

Azure Machine Learning será utilizado para:

- registrar el modelo;
- administrar una versión del modelo;

- definir sus dependencias;
- crear un deployment;
- crear un endpoint para realizar inferencias.

El endpoint generado deberá poder ser consumido desde el backend.

11. Backend

El backend será desarrollado utilizando Python y FastAPI.

FastAPI será responsable de exponer las funcionalidades principales mediante endpoints.

Como mínimo se implementarán:

GET /clientes

Consultar clientes.

GET /clientes/{id}

Consultar información de un cliente.

GET /segmentos

Consultar segmentos generados.

GET /segmentos/{id}

Consultar información de un segmento.

POST /segmentar

Ejecutar el proceso de segmentación.

POST /recomendaciones

Generar una recomendación para un segmento.

Los endpoints podrán ajustarse durante el desarrollo.

12. Integración con Azure Machine Learning

El backend deberá poder consumir el endpoint generado por Azure Machine Learning.

FastAPI deberá enviar los datos necesarios al modelo desplegado y recibir el resultado de la segmentación.

El acceso al endpoint deberá realizarse desde el backend para evitar exponer credenciales en el frontend.

13. Recomendaciones mediante Azure OpenAI Service

El MVP deberá utilizar Azure OpenAI Service para generar recomendaciones comerciales básicas.

La IA recibirá información resumida de cada segmento.

Ejemplo:

- gasto promedio;
- frecuencia promedio;
- tiempo desde última compra;
- preferencias principales.

A partir de estos datos podrá generar una recomendación comercial.

Ejemplo:

“Este segmento presenta alta frecuencia de compra y alto valor económico. Se recomienda implementar estrategias de fidelización, beneficios exclusivos y promociones personalizadas.”

Las recomendaciones estarán orientadas al segmento completo y no a clientes individuales.

14. Frontend

El frontend será desarrollado utilizando React.

La interfaz deberá ser sencilla y enfocada en demostrar las funciones principales del sistema.

Como mínimo deberá incluir:

Dashboard

Mostrar:

- número total de clientes;
- cantidad de segmentos;
- distribución de clientes por segmento.

Vista de segmentos

Mostrar:

- nombre del segmento;
- cantidad de clientes;
- características principales;
- valores promedio.

Detalle del segmento

Mostrar:

- clientes pertenecientes al segmento;
- principales métricas;
- descripción del segmento;
- recomendación comercial.

Vista de clientes

Mostrar información básica de los clientes utilizados por el sistema.

15. Requerimientos funcionales del MVP

RF-MVP-01. El sistema deberá permitir utilizar datos simulados de clientes.

RF-MVP-02. El sistema deberá aceptar información estructurada mediante CSV o JSON.

RF-MVP-03. El sistema deberá validar los datos recibidos.

RF-MVP-04. El sistema deberá limpiar y transformar los datos antes del análisis.

RF-MVP-05. El sistema deberá calcular variables relevantes para segmentación.

RF-MVP-06. El sistema deberá ejecutar un modelo de clustering.

RF-MVP-07. El sistema deberá asignar cada cliente a un segmento.

RF-MVP-08. El sistema deberá almacenar los resultados.

RF-MVP-09. El sistema deberá permitir consultar los segmentos generados.

RF-MVP-10. El sistema deberá permitir consultar los clientes pertenecientes a cada segmento.

RF-MVP-11. El sistema deberá mostrar las principales características de cada segmento.

RF-MVP-12. El sistema deberá exponer sus funcionalidades mediante FastAPI.

RF-MVP-13. El frontend deberá consumir la API desarrollada.

RF-MVP-14. El sistema deberá generar recomendaciones comerciales para los segmentos.

RF-MVP-15. El sistema deberá permitir volver a ejecutar la segmentación cuando cambien los datos.

16. Requerimientos no funcionales del MVP

RNF-MVP-01. Seguridad

Las credenciales y claves de servicios externos deberán mantenerse en el backend.

RNF-MVP-02. Privacidad

El sistema utilizará únicamente datos simulados durante el desarrollo.

RNF-MVP-03. Rendimiento

Las consultas principales deberán responder de forma adecuada para una demostración interactiva.

RNF-MVP-04. Modularidad

Frontend, backend, procesamiento de datos y Machine Learning deberán mantenerse separados.

RNF-MVP-05. Adaptabilidad

La fuente de datos simulada deberá poder ser sustituida posteriormente por una fuente real.

RNF-MVP-06. Manejo de errores

El sistema deberá responder de forma controlada ante archivos inválidos, datos incompletos o fallos de servicios externos.

RNF-MVP-07. Mantenibilidad

El código deberá mantenerse organizado por módulos y responsabilidades.

17. Tecnologías utilizadas

El MVP utilizará principalmente:

- Python.
- FastAPI.
- PostgreSQL.

- React.
- Pandas.
- NumPy.
- Scikit-learn.
- Azure Machine Learning.
- Azure OpenAI Service.

Docker podrá utilizarse para facilitar la ejecución de los componentes si el tiempo de desarrollo lo permite.

18. Funcionalidades no incluidas en el MVP

No serán obligatorias para esta primera versión:

- integración con un CRM real;
- conexión con ERP real;
- integración con Google Analytics;
- campañas reales;
- actualización de datos en tiempo real;
- creación automática de campañas;
- integración bidireccional con CRM;
- Power BI completo;
- Azure AI Search;
- Microsoft Entra ID;
- GitHub Actions;
- reentrenamiento automático;
- monitoreo avanzado del modelo;
- predicción de abandono;
- Customer Lifetime Value avanzado;
- sistema completo de roles empresariales;
- infraestructura de producción.

Estas funcionalidades podrán desarrollarse posteriormente.

19. Criterios de aceptación

El MVP se considerará terminado cuando permita completar correctamente las siguientes acciones:

1. Cargar un dataset simulado de clientes.
 2. Validar y procesar los datos.
 3. Calcular las variables necesarias para segmentación.
 4. Ejecutar el modelo de Machine Learning.
 5. Obtener al menos varios segmentos diferenciados.
 6. Asignar clientes a dichos segmentos.
 7. Almacenar los resultados en PostgreSQL.
 8. Consultar los resultados mediante FastAPI.
 9. Mostrar los segmentos desde React.
 10. Consultar los clientes pertenecientes a cada segmento.
 11. Mostrar las características principales de cada segmento.
 12. Generar una recomendación comercial mediante Azure OpenAI.
 13. Ejecutar correctamente una demostración completa del sistema.
-

20. Resultado esperado

Al finalizar el desarrollo deberá existir una aplicación funcional capaz de demostrar el concepto principal del Motor Inteligente de Segmentación de Clientes.

El usuario podrá cargar o utilizar información simulada, ejecutar el proceso de segmentación y visualizar grupos de clientes con características similares.

El sistema deberá mostrar información suficiente para comprender por qué cada segmento es diferente y proporcionar recomendaciones comerciales relacionadas con sus características.

21. Plan de desarrollo

Semana 1

- Crear dataset simulado.
- Diseñar base de datos.
- Implementar procesamiento de datos.
- Calcular variables RFM.

- Implementar primer modelo de clustering.
- Evaluar y ajustar clusters.
- Interpretar segmentos.

Semana 2

- Crear backend con FastAPI.
- Integrar PostgreSQL.
- Crear endpoints principales.
- Registrar modelo en Azure Machine Learning.
- Crear deployment.
- Crear endpoint del modelo.
- Conectar FastAPI con Azure Machine Learning.
- Integrar Azure OpenAI.

Semana 3

- Crear interfaz en React.
 - Implementar dashboard.
 - Crear vistas de clientes y segmentos.
 - Mostrar recomendaciones.
 - Integrar frontend con FastAPI.
 - Realizar pruebas.
 - Corregir errores.
 - Preparar documentación.
 - Preparar demostración final.
-

22. Prioridades del desarrollo

En caso de limitaciones de tiempo, el orden de prioridad será:

1. Dataset funcional.
2. Procesamiento de datos.
3. Modelo de segmentación.
4. Interpretación de segmentos.
5. FastAPI.
6. PostgreSQL.
7. Visualización en React.
8. Azure Machine Learning.
9. Azure OpenAI.

10. Mejoras visuales y funcionalidades adicionales.

El motor de segmentación deberá tener prioridad sobre funcionalidades secundarias.

23. Supuestos del MVP

Se considerarán los siguientes supuestos:

- No existe acceso directo a los datos reales de la empresa.
- Los datos del CRM serán simulados.
- La estructura del dataset será diseñada para representar un escenario empresarial realista.
- No será necesario modificar un CRM existente.
- No será obligatorio realizar integración con sistemas productivos.
- Los usuarios serán simulados según roles comerciales generales.
- El modelo deberá demostrar la viabilidad técnica del concepto.
- La arquitectura deberá permitir una futura integración con datos reales.
- El desarrollo deberá completarse dentro de un periodo aproximado de tres semanas.